Jhanvi Cars — Complete Local → Kubernetes Migration Runbook

All major commands, execution order, validation, troubleshooting, and the actual migration state recorded in this project

0. Environment and execution model

Ubuntu development VM: source code, Docker builds, Helm chart, Git commits/pushes.
AWS kubeadm master: kubectl access and Kubernetes/Helm deployment. The recorded AWS master repository is `~/jhanvi-cars-k8s`.

```
# VM
cd ~/jhanvi-dev

# AWS master
cd ~/jhanvi-cars-k8s
```

1. Pre-migration application inventory

Confirm the microservices and ports before containerization.

```
API Gateway       3000
Auth service      3001
User service      3002
Driver service    3003
Booking service   3004
Vehicle service   3005
Payment service   3006
Notification     3007
Frontend          80
PostgreSQL       5432
Memcached       11211
```

2. Git workflow — create migration branch

Keep main stable and perform Kubernetes migration work on a feature branch.

```
cd ~/jhanvi-dev
git checkout main
git pull origin main
git checkout -b feature/kubernetes-migration
git status
git branch --show-current
```

3. Build backend Docker images

Build each service from its service directory. The project uses versioned backend images with tag 1.0.

```
cd ~/jhanvi-dev

docker build -t jhanviadmin/jhanvi-cars-api-gateway:1.0 ./backend/api-gateway
docker build -t jhanviadmin/jhanvi-cars-auth-service:1.0 ./backend/auth-service
docker build -t jhanviadmin/jhanvi-cars-user-service:1.0 ./backend/user-service
docker build -t jhanviadmin/jhanvi-cars-driver-service:1.0 ./backend/driver-service
docker build -t jhanviadmin/jhanvi-cars-booking-service:1.0 ./backend/booking-service
docker build -t jhanviadmin/jhanvi-cars-vehicle-service:1.0 ./backend/vehicle-service
docker build -t jhanviadmin/jhanvi-cars-payment-service:1.0 ./backend/payment-service
docker build -t jhanviadmin/jhanvi-cars-notification-service:1.0 ./backend/notification-service

docker images | grep jhanviadmin
```

4. Build frontend image

The frontend Vite build uses build-time browser configuration. The recorded approach uses BuildKit secret mounting for `.env.local`.

```
cd ~/jhanvi-dev/frontend

DOCKER_BUILDKIT=1 docker build \
  --secret id=frontend_env,src=.env.local \
  -t jhanviadmin/jhanvi-cars-frontend:latest .

docker images | grep jhanvi-cars-frontend
```

5. Push Docker images

```
docker login

docker push jhanviadmin/jhanvi-cars-api-gateway:1.0
docker push jhanviadmin/jhanvi-cars-auth-service:1.0
docker push jhanviadmin/jhanvi-cars-user-service:1.0
docker push jhanviadmin/jhanvi-cars-driver-service:1.0
docker push jhanviadmin/jhanvi-cars-booking-service:1.0
docker push jhanviadmin/jhanvi-cars-vehicle-service:1.0
docker push jhanviadmin/jhanvi-cars-payment-service:1.0
docker push jhanviadmin/jhanvi-cars-notification-service:1.0
docker push jhanviadmin/jhanvi-cars-frontend:latest
```

6. Create Helm chart

```
cd ~/jhanvi-dev
mkdir -p helm
cd helm
helm create jhanvi-cars

# Inspect
find jhanvi-cars -maxdepth 2 -type f -print
helm version
```

The project then replaced the generated chart content with the Jhanvi Cars chart containing Deployments, Services, ConfigMap, Secrets references, PostgreSQL, Memcached, migration Jobs, frontend, Ingress, and namespace resources.

7. Helm chart validation

```
cd ~/jhanvi-dev/helm/jhanvi-cars

helm lint .

helm template jhanvi-cars . >/tmp/jhanvi-cars-rendered.yaml

# Inspect rendered resources
grep '^kind:' /tmp/jhanvi-cars-rendered.yaml

# Inspect images
grep 'image:' /tmp/jhanvi-cars-rendered.yaml

# Verify PostgreSQL grants
grep 'GRANT USAGE' /tmp/jhanvi-cars-rendered.yaml
```

Expected lint result: 1 chart linted, 0 charts failed. The final grant fix changed service schema permissions from USAGE only to USAGE plus CREATE.

8. Kubernetes configuration used by the chart

```
AUTH_SERVICE_URL=http://auth-service:3001
USER_SERVICE_URL=http://user-service:3002
DRIVER_SERVICE_URL=http://driver-service:3003
BOOKING_SERVICE_URL=http://booking-service:3004
VEHICLE_SERVICE_URL=http://vehicle-service:3005
PAYMENT_SERVICE_URL=http://payment-service:3006
NOTIFICATION_SERVICE_URL=http://notification-service:3007
MEMCACHED_HOST=memcached
MEMCACHED_PORT=11211
MEMCACHED_TTL_SECONDS=60
PROXY_TIMEOUT_MS=5000
NODE_ENV=production
```

Kubernetes DNS replaces local `localhost`. Services are ClusterIP services.

9. PostgreSQL design and initialization

```
Database: jhanvi_cars
Bootstrap user: jhanvi_admin

Schemas:
auth_schema
user_schema
driver_schema
booking_schema
vehicle_schema
payment_schema
notification_schema

Application roles:
auth_user
user_user
```

The PostgreSQL StatefulSet mounts a persistent volume and the initialization ConfigMap into `/docker-entrypoint-initdb.d`. The script creates the extension, schemas, roles, and grants.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE SCHEMA IF NOT EXISTS auth_schema;
CREATE SCHEMA IF NOT EXISTS user_schema;
CREATE SCHEMA IF NOT EXISTS driver_schema;
CREATE SCHEMA IF NOT EXISTS booking_schema;
CREATE SCHEMA IF NOT EXISTS vehicle_schema;
CREATE SCHEMA IF NOT EXISTS payment_schema;
CREATE SCHEMA IF NOT EXISTS notification_schema;

GRANT USAGE, CREATE ON SCHEMA user_schema TO user_user;
GRANT USAGE, CREATE ON SCHEMA driver_schema TO user_user;
GRANT USAGE, CREATE ON SCHEMA booking_schema TO user_user;
GRANT USAGE, CREATE ON SCHEMA vehicle_schema TO user_user;
GRANT USAGE, CREATE ON SCHEMA payment_schema TO user_user;
GRANT USAGE, CREATE ON SCHEMA notification_schema TO user_user;
GRANT USAGE, CREATE ON SCHEMA auth_schema TO auth_user;
```

10. Kubernetes Secret preparation

Create the application/database Secret on the cluster. Do not commit actual values to Git. The recorded Secret contained database passwords, database URLs, internal service token, JWT secret, and Razorpay credentials.

```
# Example structure — replace placeholders with the real values securely
kubectl create secret generic jhanvi-cars-secrets \
  -n jhanvi-cars \
  --from-literal=POSTGRES_PASSWORD='<POSTGRES_PASSWORD>' \
  --from-literal=USER_DB_PASSWORD='<USER_DB_PASSWORD>' \
  --from-literal=AUTH_DB_PASSWORD='<AUTH_DB_PASSWORD>' \
  --from-literal=DATABASE_URL='postgresql://user_user:<USER_DB_PASSWORD>@postgres:5432/jhanvi_cars' \
  --from-literal=AUTH_DATABASE_URL='postgresql://auth_user:<AUTH_DB_PASSWORD>@postgres:5432/jhanvi_cars' \
  --from-literal=INTERNAL_SERVICE_TOKEN='<TOKEN>' \
  --from-literal=JWT_SECRET='<JWT_SECRET>' \
  --from-literal=RAZORPAY_KEY_ID='<KEY_ID>' \
  --from-literal=RAZORPAY_KEY_SECRET='<KEY_SECRET>' \
  --dry-run=client -o yaml | kubectl apply -f -

kubectl get secret jhanvi-cars-secrets -n jhanvi-cars
kubectl describe secret jhanvi-cars-secrets -n jhanvi-cars
```

The actual project setup generated random database passwords and transferred temporary credential files securely, then removed the temporary files. Those values are intentionally absent from this document.

11. Commit Helm work on VM

```
cd ~/jhanvi-dev/helm/jhanvi-cars

git status
git add .
git commit -m "feat: add helm deployment for jhanvi cars"
git push origin feature/kubernetes-migration

# Later permission fix:
git add templates/postgres-init.yaml
git commit -m "fix: allow service users to create schema objects"
git push origin feature/kubernetes-migration
```

Recorded final migration-permission commit: `233496b`.

12. Synchronize AWS master from GitHub

```
cd ~/jhanvi-cars-k8s

git status
git branch --show-current

git fetch origin
git pull --ff-only origin feature/kubernetes-migration

git log -1 --oneline
git status
```

Expected final commit at the recorded stage: `233496b fix: allow service users to create schema objects`, with a clean working tree.

13. Namespace setup / Helm ownership

The first Helm install failed because `jhanvi-cars` already existed without Helm ownership metadata. The existing namespace was adopted by Helm.

```
kubectl get namespace jhanvi-cars

kubectl label namespace jhanvi-cars \
  app.kubernetes.io/managed-by=Helm --overwrite

kubectl annotate namespace jhanvi-cars \
  meta.helm.sh/release-name=jhanvi-cars \
  meta.helm.sh/release-namespace=jhanvi-cars \
  --overwrite

kubectl get namespace jhanvi-cars --show-labels
```

14. Deploy with Helm

```
cd ~/jhanvi-cars-k8s

helm upgrade --install jhanvi-cars ./helm/jhanvi-cars \
  --namespace jhanvi-cars \
  --create-namespace

helm status jhanvi-cars -n jhanvi-cars
helm list -n jhanvi-cars
```

Recorded successful result: STATUS deployed, REVISION 1.

15. Immediate Kubernetes verification

```
kubectl get all -n jhanvi-cars
kubectl get pvc -n jhanvi-cars
kubectl get ingress -n jhanvi-cars
kubectl get jobs -n jhanvi-cars

kubectl get pods -n jhanvi-cars -o wide
```

16. Verify PostgreSQL

```
kubectl get pod postgres-0 -n jhanvi-cars
kubectl describe pod postgres-0 -n jhanvi-cars
kubectl logs postgres-0 -n jhanvi-cars

kubectl get pvc -n jhanvi-cars
kubectl get svc postgres -n jhanvi-cars
```

Wait for PostgreSQL to become `1/1 Running` before treating migration failures as application failures.

17. Verify Memcached

```
kubectl get pod -l app=memcached -n jhanvi-cars
kubectl logs deployment/memcached -n jhanvi-cars
kubectl get svc memcached -n jhanvi-cars
```

18. Verify backend services

```
kubectl get deployments -n jhanvi-cars

kubectl get pods -n jhanvi-cars

kubectl describe deployment api-gateway -n jhanvi-cars
kubectl describe deployment auth -n jhanvi-cars
kubectl describe deployment user -n jhanvi-cars
kubectl describe deployment driver -n jhanvi-cars
kubectl describe deployment booking -n jhanvi-cars
kubectl describe deployment vehicle -n jhanvi-cars
kubectl describe deployment payment -n jhanvi-cars
kubectl describe deployment notification -n jhanvi-cars
```

19. Verify Services and DNS names

```
kubectl get svc -n jhanvi-cars

# Expected important names:
# api-gateway-service
# auth-service
# user-service
# driver-service
# booking-service
```

```
# vehicle-service
# payment-service
# notification-service
# frontend
# postgres
# memcached
```

20. Verify Ingress

```
kubectl get ingress -n jhanvi-cars
kubectl describe ingress -n jhanvi-cars

# If an ingress controller service/IP is available:
kubectl get svc -A | grep -i ingress
```

The chart routes `/api` to the API Gateway and the root path to the frontend.

21. Migration Jobs

```
kubectl get jobs -n jhanvi-cars
kubectl get pods -n jhanvi-cars | grep migration

kubectl describe job user-migration -n jhanvi-cars
kubectl describe job auth-migration -n jhanvi-cars
kubectl describe job driver-migration -n jhanvi-cars
kubectl describe job booking-migration -n jhanvi-cars
kubectl describe job vehicle-migration -n jhanvi-cars
kubectl describe job payment-migration -n jhanvi-cars
kubectl describe job notification-migration -n jhanvi-cars
```

22. Migration troubleshooting — when pods disappear

```
kubectl logs job/user-migration -n jhanvi-cars

# If this times out because the failed pod was deleted:
kubectl describe job user-migration -n jhanvi-cars

kubectl get events -n jhanvi-cars --sort-by='.lastTimestamp' | tail -30
```

The recorded cluster events showed migration containers starting and then entering BackOff, followed by Jobs reaching BackoffLimitExceeded. The original failed pods were deleted.

23. Reproduce a migration manually

When the Job has deleted its failed pod, create a temporary debug pod with the same image, ConfigMap, Secret, schema, and migration command.

```
cat > /tmp/user-migration-debug.yaml <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: user-migration-debug
  namespace: jhanvi-cars
spec:
  restartPolicy: Never
  containers:
    - name: user-migration
      image: jhanviadmin/jhanvi-cars-user-service:1.0
      command: ["npm", "run", "migrate"]
      envFrom:
        - configMapRef:
            name: jhanvi-cars-config
        - secretRef:
            name: jhanvi-cars-secrets
      env:
        - name: DB_SCHEMA
          value: "user_schema"
EOF

kubectl apply -f /tmp/user-migration-debug.yaml
kubectl get pod user-migration-debug -n jhanvi-cars
kubectl logs user-migration-debug -n jhanvi-cars
```

24. Actual migration failure discovered

The debug migration produced the following error in the recorded run:

```
> user-service@1.0.0 migrate
> knex --knexfile knexfile.ts migrate:latest
```

```
Failed to load external module ts-node/register
Failed to load external module typescript-node/register
...
Unexpected token '{'
SyntaxError: Unexpected token '{'
```

Interpretation: the migration command is trying to load `knexfile.ts`, but the production image does not have a working TypeScript runtime/loader for that configuration. This occurs before the intended database migration completes.

25. Inspect the service before fixing the image

```
# Development VM
cd ~/jhanvi-dev

cat backend/user-service/package.json
cat backend/user-service/Dockerfile
cat backend/user-service/knexfile.ts

# Then inspect other services for the same pattern:
cat backend/auth-service/package.json
cat backend/driver-service/package.json
cat backend/booking-service/package.json
cat backend/vehicle-service/package.json
cat backend/payment-service/package.json
cat backend/notification-service/package.json
```

The migration packaging should be fixed consistently across all services. Do not change PostgreSQL credentials merely because the migration Job failed with this TypeScript loader error.

26. Recommended migration-runtime validation loop

```
# On VM after fixing one service
docker build -t jhanviadmin/jhanvi-cars-user-service:1.0 ./backend/user-service
docker push jhanviadmin/jhanvi-cars-user-service:1.0

# On AWS master
kubectl delete pod user-migration-debug -n jhanvi-cars --ignore-not-found

# Recreate the debug pod or update the Job/image as appropriate
kubectl get pods -n jhanvi-cars
kubectl logs <migration-pod> -n jhanvi-cars

# After successful validation, rebuild/push the remaining affected service images.
```

For a compiled production image, one clean approach is to use compiled JavaScript Knex configuration/migrations. Another approach is to include and correctly configure the TypeScript runtime. The project had not yet selected/implemented the final option at the recorded stopping point.

27. Database verification after successful migrations

```
# Discover PostgreSQL pod
kubectl get pods -n jhanvi-cars -l app=postgres

# Open psql using the bootstrap/admin credentials if available securely
kubectl exec -it postgres-0 -n jhanvi-cars -- psql -U jhanvi_admin -d jhanvi_cars

# Inside psql:
\dn
\du
\dt auth_schema.*
\dt user_schema.*
\dt driver_schema.*
\dt booking_schema.*
\dt vehicle_schema.*
\dt payment_schema.*
\dt notification_schema.*
\q
```

Use the appropriate secure database password when prompted. Never put real passwords into this runbook or Git.

28. Application smoke test

```
# Kubernetes status
kubectl get pods -n jhanvi-cars
kubectl get svc -n jhanvi-cars
kubectl get ingress -n jhanvi-cars
kubectl get jobs -n jhanvi-cars

# API Gateway logs
```

```
kubectl logs deployment/api-gateway -n jhanvi-cars

# Service logs
kubectl logs deployment/auth -n jhanvi-cars
kubectl logs deployment/user -n jhanvi-cars
kubectl logs deployment/driver -n jhanvi-cars
kubectl logs deployment/booking -n jhanvi-cars
kubectl logs deployment/vehicle -n jhanvi-cars
kubectl logs deployment/payment -n jhanvi-cars
kubectl logs deployment/notification -n jhanvi-cars
```

Then exercise the real application flow: frontend → Ingress → API Gateway → backend service → PostgreSQL, including registration/login, driver/vehicle operations, booking, payment, and notification. Vehicle-service should use Memcached through the `memcached` Service.

29. General Kubernetes troubleshooting commands

```
kubectl get pods -n jhanvi-cars -o wide
kubectl get pods -n jhanvi-cars --show-labels
kubectl describe pod <pod> -n jhanvi-cars
kubectl logs <pod> -n jhanvi-cars
kubectl logs <pod> -n jhanvi-cars --previous
kubectl get events -n jhanvi-cars --sort-by='.lastTimestamp'
kubectl get endpoints -n jhanvi-cars
kubectl get endpointslices -n jhanvi-cars
kubectl get configmap jhanvi-cars-config -n jhanvi-cars -o yaml
kubectl get secret jhanvi-cars-secrets -n jhanvi-cars
kubectl rollout status deployment/<name> -n jhanvi-cars
kubectl rollout restart deployment/<name> -n jhanvi-cars
```

30. Helm lifecycle commands

```
helm list -n jhanvi-cars
helm status jhanvi-cars -n jhanvi-cars
helm get values jhanvi-cars -n jhanvi-cars
helm get manifest jhanvi-cars -n jhanvi-cars
helm history jhanvi-cars -n jhanvi-cars

# Upgrade after chart/image changes
helm upgrade jhanvi-cars ./helm/jhanvi-cars -n jhanvi-cars

# Roll back a bad Helm revision if required
helm rollback jhanvi-cars <REVISION> -n jhanvi-cars
```

31. Important cleanup commands — use deliberately

```
# Remove only the temporary debug pod
kubectl delete pod user-migration-debug -n jhanvi-cars --ignore-not-found

# If this is a disposable practice environment and you intentionally want
# to remove the whole Helm release:
helm uninstall jhanvi-cars -n jhanvi-cars

# Namespace removal is destructive:
kubectl delete namespace jhanvi-cars
```

Do not delete the namespace/PVC during troubleshooting unless you intentionally want to destroy the practice environment and database data.

32. Final Git flow after Kubernetes validation

```
# VM
cd ~/jhanvi-dev
git status

git checkout main
git pull origin main

# Merge only after feature validation
git merge --no-ff feature/kubernetes-migration
git push origin main

# Optional cleanup after merge
git branch -d feature/kubernetes-migration
git push origin --delete feature/kubernetes-migration
```

Follow the project's controlled feature-branch workflow. Do not merge unfinished or unverified Kubernetes changes.

33. Final completion checklist

```
[ ] All Docker images build successfully
[ ] All required images are pushed
[ ] Helm lint passes
[ ] Helm template renders successfully
[ ] ConfigMap contains Kubernetes DNS names
[ ] Secret exists and real values are not in Git
[ ] PostgreSQL StatefulSet is Ready
[ ] PostgreSQL PVC is Bound
[ ] PostgreSQL schemas/users/privileges are correct
[ ] Memcached is Ready
[ ] All backend Deployments are Ready
[ ] Frontend Deployment is Ready
[ ] ClusterIP Services exist
[ ] Ingress exists and routes correctly
[ ] All migration Jobs complete successfully
[ ] Database tables exist
[ ] Frontend login/registration works
[ ] Driver/vehicle flow works
[ ] Booking works
[ ] Payment works
[ ] Notification works
[ ] Memcached caching works
[ ] Git branch is validated and ready for merge
```

34. Recorded state at the time of this runbook

Completed: containerization, Docker image publishing, Helm chart, Helm validation, Kubernetes Secret setup, PostgreSQL StatefulSet/PVC design, PostgreSQL initialization, Memcached, application Deployments, Services, Ingress configuration, Git feature-branch workflow, AWS Git synchronization, and successful Helm release installation.

Remaining: fix the migration runtime packaging/configuration. The recorded user migration failure is Knex attempting to load `knexfile.ts` without a working TypeScript loader. After fixing this consistently, rerun all migration Jobs and perform the complete application smoke test.

This document intentionally records the actual stopping point and does not claim the migration is fully complete while migration Jobs remain unresolved.